SESSION 2

Brownfield AI Development

80% of enterprise work is brownfield. Understand before you change.

Pasi Vuorio | Founder & AI Architect, ModernPath

Today's Agenda

01 **Greenfield vs Brownfield** — 10 min

Different challenges, different discipline -- vibe coding is dangerous here

02 **The 4-Step Process (AGENTS.md)** — 5 min

Document -> Spec -> Develop -> Audit -- defined in your project config

03 **3-Tiered Documentation & Navigation** — 25 min

Tier structure, content-plan.md, navigation strategies, sub-agents

04 **Grounded Spec-Driven Development** — 15 min

Real spec template from AGENTS.md -- grounded in docs, not assumptions

05 **Review, Audit & Fix Workflows** — 15 min

6-pass review, audit gate, fix workflow with 3-attempt rule

Greenfield vs Brownfield

In greenfield you define -- in brownfield you must discover first

Aspect	Greenfield	Brownfield
Context	You define it	You must discover it
Architecture	Design for AI	Adapt AI to existing patterns
Risk	Low (fresh start)	High (breaking changes)
AI Strength	Generation	Understanding first, then changes
Documentation	Write as you build	Must create retroactively

Key insight: vibe coding a change to a 114k-line codebase is dangerous. You MUST understand before you change.

Vibe Coding vs Spec-Driven

Understanding the tradeoff in brownfield context

Vibe Coding

Speed	Fast start
Planning	Minimal
Documentation	None / minimal
Maintainability	Low
Best for	Prototypes, experiments

Spec-Driven (AGENTS.md)

Speed	Slower start, faster finish
Planning	Thorough (4-step process)
Documentation	3-tiered, navigable
Maintainability	High (grounded specs)
Best for	Production, brownfield

The 4-Step Process

Document -> Spec -> Develop -> Audit -- defined as workflows in AGENTS.md

1

Document

Scan filesystem vs plan.
Generate 3-tiered docs.
Update content-plan.md.

`research + document`

2

Spec

Create grounded spec.
ACs, files to modify,
risk + testing strategy.

`spec workflow`

3

Develop

TDD: Red -> Green ->
Refactor. Implement to
spec, update docs.

`tdd + develop`

4

Audit

6-pass review, then
final audit gate.
Fix workflow if needed.

`review + audit + fix`

This loop repeats for every change. Documentation builds up over time, making each cycle faster.

Rehearsal: Orient to Suroi

A real 114k LOC TypeScript 2D battle royale -- your brownfield exercise codebase

The Codebase

`client/` ~30k LOC -- rendering, input, UI

`server/` ~45k LOC -- game sim, networking

`common/` ~25k LOC -- shared types, defs

Live demo: suroi.io

AGENTS.md Structure

Part 1	Documentation Plan
Part 2	Spec-Driven Development
Part 3	Skills & Tools

10 min

3-Tiered Documentation

Each tier references the others -- navigation up, down, and sideways

TIER 1 -- Architecture

`docs/`

Purpose, tech stack, system boundaries, data model.

Navigation: Tier 1 -> Tier 2 via subsystem references

`architecture.md, datamodel.md, protocol.md`

TIER 2 -- Subsystems

`docs/subsystems/<name>/`

Domain-specific subsystems, interfaces, data flow.

Navigation: up Tier 1 | down Tier 3 | across subsystems

`README.md, patterns.md, modules/`

TIER 3 -- Modules & Code

`docs/subsystems/<name>/modules/`

Implementation patterns, business rules, @file refs.

Navigation: up Tier 2 | -> source code references

`<module>.md with @file links to src/`

content-plan.md

The index of all documentation -- AI reads this first

Documentation Index -- Suroi

#	Module / Area	Tier	Status	Path	Priority
1	Architecture Overview	T1	Not Started	docs/architecture.md	Critical
2	Game Data Model	T1	Not Started	docs/datamodel.md	Critical
3	Protocol Reference	T1	Not Started	docs/protocol.md	Critical
5	Definitions Subsystem	T2	Not Started	docs/subsystems/definitions/	Critical
6	Packets Subsystem	T2	Not Started	docs/subsystems/packets/	Critical
8	Game Loop Subsystem	T2	Not Started	docs/subsystems/game-loop/	High
9	Inventory Subsystem	T2	Not Started	docs/subsystems/inventory/	High
11	Rendering Subsystem	T2	Not Started	docs/subsystems/rendering/	High

Generation order: Tier 1 first -> Core T2 (Definitions, Packets, Objects) -> Server T2 -> Client T2 -> T3 last

Documentation Navigation

Three strategies AI uses to navigate your codebase docs

Top-Down

Tier 1 -> Tier 2 -> Tier 3 -> Code.
Use when: understanding structure, planning changes

Feature-Wise

content-plan -> spec -> affected subsystems -> modules -> code.
Use when: implementing features

Package-Wise

common/definitions -> common/packets -> server/objects -> client/rendering.
Use when: debugging sync

Navigation Rules from AGENTS.md

1. Start with content-plan.md 2. Use all three strategies 3. Follow cross-tier refs 4. Follow @file refs 5. Trace across packages 6. Update docs when code changes

Sub-Agent Definitions

One agent per workflow step -- each follows its AGENTS.md workflow

Documentation Agent

Runs the 'document' workflow

1. Read content-plan.md
2. Scan filesystem vs plan
3. Generate/update 3-tiered docs
4. Update content-plan.md

Can: Read all source, write docs/ only

Cannot: Edit source code or tests

Output: Scan report + updated docs

Spec Agent

Runs the 'spec' workflow

1. Study docs (grounding)
2. Create spec from template
3. Fill ACs, files, risk, tests
4. Run readiness checklist

Can: Read docs + code, write specs/

Cannot: Implement code or run tests

Output: Grounded spec ready for dev

Review Agent

Runs the 'review' + 'audit' workflows

1. Spec compliance audit
2. Test coverage audit
3. Security + architecture audit
4. Performance audit + verdict

Can: Read everything, write reports

Cannot: Modify implementation code

Output: Review report + PASS/FAIL

Each agent maps to a workflow in AGENTS.md Part 2. Single Responsibility -- the doc agent never writes code, the spec agent never implements.

Activity: Walk the Doc Process

Use the pregenerated bundle -- map, navigate, configure agents

1. **Map**
   Trace each row in content-plan.md to the matching source directory

2. **Navigate**
   Use all 3 strategies: top-down (Tier 1->2->3), feature-wise, package-wise

3. **Configure**
   Paste a sub-agent definition into AGENTS.md with Can/Cannot boundaries

10 min

The Spec Template

From AGENTS.md -- grounded in documentation, not hallucinations

specs/features/<name>.md

Overview	Status, packages, subsystems, protocol change?
Problem Statement	What problem? Why needed?
Current Behavior	Reference Tier 2/3 docs
Proposed Change	What it should do after
Acceptance Criteria	Given / When / Then per AC
Files to Modify	Package | File | Change
Risk Assessment	What breaks, balance, perf, rollback
Testing Strategy	Unit + integration + E2E + coverage
Related Documentation	Links to all 3 tiers (mandatory)

Ungrounded Spec

"Add caching to the API"
No context, no doc refs, no affected subsystems, no test plan. AI invents a solution from imagination.

Grounded Spec

Context: see docs/subsystems/game-loop/
ACs with Given/When/Then
Files: common/src/objects/player.ts
Risk: affects UpdatePacket serialization
Tests: unit + integration + playtesting
Docs: links Tier 1, 2, 3

Build: Write a Grounded Spec

Use prompts/03-spec-changes.md -- specs grounded in the pregenerated documentation

1. **Pick an improvement for a Suroi subsystem**

   A real issue, balance change, or new feature -- reference content-plan.md

2. **Fill the spec template with grounded evidence**

   Current behavior from docs, files to modify, ACs with Given/When/Then, testing strategy

3. **Run the readiness checklist**

   Every AC testable? Files listed? Risk assessed? Testing comprehensive? All 3 tiers linked?

10 min

Review, Audit & Fix Workflows

From AGENTS.md -- multi-pass review, final audit gate, systematic fixing

Review: 6 Audit Passes

1. Spec Compliance	Every requirement met?
2. Test Coverage	Spec tests vs actual tests
3. Security Audit	Input validation, auth, secrets
4. Architecture	Patterns, dependencies, data flow
5. Performance	Queries, caching, bundle size
6. Final Verdict	APPROVE | REQUEST CHANGES | BLOCK

Audit: Final Gate

1. Completeness check (all ACs met?)
2. Run full review workflow
3. Documentation audit (docs updated?)
4. Final test run (coverage met?)

docs/learnings.md

Check FIRST when encountering errors. Add solutions so the same problem is never debugged twice. Template: Context / Symptom / Root Cause / Fix

Fix Workflow (3-Attempt Rule: never loop more than 3 times without human input)

Gather Context → Hypothesize → Validate → Fix → Document

Build: Review & Audit

Use prompts/04-implement.md -- implement, review, audit

1. **Run the 6-pass review on your spec change**
   Spec compliance, test coverage, security, architecture, performance, verdict

2. **Run the audit gate**
   All ACs met? Docs updated? Coverage target hit? PASS / FAIL / REQUIRES ADJUSTMENT

3. **If issues found, run the fix workflow**
   Gather context -> Hypothesize -> Validate -> Fix -> Document in learnings.md

10 min

Key Takeaways

- Brownfield = understand before you change. AGENTS.md is your process bible.
- Document -> Spec -> Develop -> Audit -- 4-step loop defined as named workflows
- 3-tiered docs with cross-references -- AI navigates up, down, and across tiers
- Sub-agents with Can/Cannot -- single responsibility, non-overlapping roles
- Ground specs in evidence -- reference real docs, real code, real test plans
- 6-pass review + audit gate + fix workflow with 3-attempt rule + learnings.md

Next: Session 3 -- Advanced Agentic Workflows (MCP, CLI tools, hooks, scaling)